You

I currently have a power automate flow that links to my Canvas power app through a button link. One of my button links from the email is meant to take me to a specific page of the power app but it takes me to the initial landing page every time, the way I have specified this is by an additional parameter in the url that can be detected by the power app. My question is that how can I set it up in the power app so that it automatically navigates to the specified page if the url parameter is present and stays on the initial page otherwise? Please can you specify what formulas I need and where to place them so that no user action is required to facilitate this and nothing goes wrong

Copilot

To achieve this, you can use the *Navigate* function in Power Apps to check for the URL parameter and navigate to the appropriate page.[1] Here's how you can set it up:

1. **Create a parameter in your Power App**: Add a parameter to your Power App that will capture the URL parameter.[1]
2. **Set up the *OnStart* property**: Use the *OnStart* property of the App to check for the parameter and navigate accordingly.[1]

Here's an example of how you can do this:

1. **Add a parameter**: In your Power App, go to the settings and add a parameter (e.g., *MyParameter*).
2. **Set up the *OnStart* property**: In the App's properties, find the *OnStart* property and add the following formula:

```
If(
    Param("MyParameter") <> "",
    Navigate(SpecificPage),
    Navigate(LandingPage)
)
```

Replace *MyParameter* with the name of your parameter, *SpecificPage* with the name of the page you want to navigate to if the parameter is present, and *LandingPage* with the name of your initial landing page.

This formula checks if the parameter is present in the URL. If it is, it navigates to the *SpecificPage*; otherwise, it navigates to the *LandingPage*.

Does this help with what you're trying to achieve?

References

(1) New Dynamic LLC

You

(Now Wednesday, Feb 5)
This is my code in *OnStart*, however I get an error:

```
If(
   Param("page") <> "",
   Navigate(thisRequest),
   Navigate(allRequests)
)
```

Navigate is not permitted in *OnStart*. Use the *StartScreen* property instead.

Copilot

You're right, Navigate is not allowed in the *OnStart* property. Instead, you should use the *App.StartScreen* property to achieve the same result. Here's how you can set it up:

1. **Set the *StartScreen* property**: Go to the App settings and find the *StartScreen* property. You can set this property to a formula that determines which screen to display based on the URL parameter.

Here's an example of how you can do this:

1. **Set the StartScreen property**:

```
If(
    Param("page") <> "",
    ThisRequest,  // Replace with your specific page
    AllRequests  // Replace with your initial landing page
)
```

This formula will check for the URL parameter page. If it's present, it navigates to *ThisRequest*. Otherwise, it navigates to *AllRequests*.

Now your app should automatically navigate to the specified page if the URL parameter is present and stay on the initial page otherwise, without any user action required.

Does this address the issue?

You

I also have another parameter called title in my url, I want to write this code or similar somewhere but I'm unsure where to put it:

```
If(
    Param("title") <> "",
    requestID = Param("title"),
)
```

It needs to link to this code from my gallery:
LookUp('SAP requests list', Title = requestID)

Copilot

To achieve this, you can use the *OnVisible* property of the screen that contains the gallery. This property executes when the screen becomes visible and allows you to set variables based on parameters.

Here's how you can set it up:

1. **Set a variable on screen load**: In the *OnVisible* property of the screen that contains the gallery, set the *requestID* variable based on the URL parameter.
2. **Use the variable in the gallery's *Items* property**: Update the *Items* property of the gallery to use the *requestID* variable.

Here's an example of how you can do this:

1. **Set the *OnVisible* property** of the screen containing the gallery:
   OnVisible = Set(requestID, Param("title"))
2. **Update the *Items* property** of the gallery to use the *requestID* variable:
   Items = LookUp('SAP requests list', Title = requestID)

With this setup, when the screen becomes visible, it will check for the *title* parameter in the URL and set the *requestID* variable. The gallery will then use this variable to look up the corresponding items.

Does this help with your requirements?

You

Now the *requestID* doesn't work when it's not in the parameter, currently in my app it gets set on a button press from the main landing page when you navigate to the *thisRequest* page

Copilot

To make sure the *requestID* works both when set from the URL parameter and from the button press on the landing page, you can initialize the *requestID* variable on screen load and then update it based on the button press if necessary.

Here's how you can do this:

1. **Initialize the *requestID* variable on screen load**: In the *OnVisible* property of the screen that contains the gallery, set the *requestID* variable based on the URL parameter.
2. **Set the *requestID* variable when the button is pressed**: Update the button's *OnSelect* property to set the *requestID* variable before navigating to the *thisRequest* page.

Here's how you can achieve this:

1. **Set the *OnVisible* property** of the screen containing the gallery:
   OnVisible = Set(requestID, If(Param("title") <> "", Param("title"), requestID))
2. **Update the button's *OnSelect* property**:
   OnSelect = Set(requestID, [YourValueHere]); Navigate(thisRequest)

Replace *[YourValueHere]* with the value you want to set for the *requestID* variable when the button is pressed.

This way, the *requestID* variable will be initialized based on the URL parameter when the screen loads, and it will be updated when the button is pressed if necessary.

Does this help resolve the issue?

You

(Now Tuesday, Feb 11)
When I put this in a text label:
If(Param("title"), Param("title"), "No param title")
I get the following output:
"No param title"
However the url is: https://apps.powerapps.com/play/e/0713ee51-a2c3-420b-bf1f-61553c866fc1/a/d8b1c96a-4d0e-4571-8d56-1b5f4fbc2658?tenantId=28b8dfd0-aa16-412c-9b26-b845b9acd1a9&source=SharePoint&list=8a8feb26-8fbb-4870-a648-89f2312941f0&item=16&page=thisRequest&title=21#

Copilot

It looks like there's an issue with how the *If* function is being evaluated. The *If* function should check for the presence of the parameter and return the parameter's value if it exists, otherwise return "No param title."

Try updating the formula to use the *IsBlank* function to check if the parameter is present. Here's the updated formula for the text label:

```
If(
    !IsBlank(Param("title")),
    Param("title"),
    "No param title"
)
```

This formula will check if the *title* parameter is not blank (*!IsBlank(Param("title"))*). If it's not blank, it will display the value of the *title* parameter. Otherwise, it will display "No param title."

Give this a try and see if it resolves the issue. If the problem persists, let me know, and we can explore further!

You

Now I have *Set(requestID, If(!IsBlank(Param("title")) && requestID = "", Param("title"), requestID))* in the *OnVisible* property of the *thisRequest* screen (the one that's automatically navigated to if 'page' is set as a parameter) but the gallery is just blank when the url is https://apps.powerapps.com/play/e/0713ee51-a2c3-

420b-bf1f-61553c866fc1/a/d8b1c96a-4d0e-4571-8d56-1b5f4fbc2658?tenantId=28b8dfd0-aa16-412c-9b26-b845b9acd1a9&source=SharePoint&list=8a8feb26-8fbb-4870-a648-89f2312941f0&item=16&page=thisRequest&title=21
The previously mentioned text box shows 21 with the code:

```
If(
    !IsBlank(Param("title")),
    Param("title"),
    "No param title"
)
```

Copilot

Thanks for clarifying! It sounds like the requestID is getting set correctly, but there might be an issue with how the gallery's items are being filtered.

1. **Ensure the OnVisible property sets the requestID correctly**:
   Set(requestID, If(!IsBlank(Param("title")) && IsBlank(requestID), Param("title"), requestID))
2. **Check the Items property of the gallery**:
   Ensure that the gallery's Items property is correctly referencing the requestID variable:
   Items = Filter('SAP requests list', Title = requestID)
   If requestID is not blank, this should filter the gallery items accordingly.
3. **Check the Default property of the form (if applicable)**:
   If you have a form on the screen, ensure the Default property is set correctly to use the requestID:
   Default = LookUp('SAP requests list', Title = requestID)
4. **Debugging**:
   You can add a temporary label to your screen to display the value of requestID to ensure it is being set correctly:
   Text = requestID

These steps should help you identify any issues with the gallery items being blank. If you still encounter problems, you might want to review the data source and ensure that the Title field contains the values you expect. Let me know how it goes!

You

(Now Wednesday, Feb 12)
In Power Automate, I have an email of a 'Requestor' and I want to retrieve their details on the system (just display name), how can I do this?

Copilot

To retrieve the display name of the requestor from their email address in Power Automate, you can use the **"Get user profile (V2)"** action.[1] Here's a step-by-step guide to help you set this up:

1. **Create a new automated flow**: Open Power Automate and sign in with your Microsoft credentials.[1] Go to **My flows** (from the left navigation), expand **+ New flow**, and select **Automated cloud flow** under the **Build your own from blank** section.[1]
2. **Choose the trigger**: Select the trigger **When a new email arrives (V3)**.[1] This will start the flow whenever a new email is received.[1]
3. **Add the "Get user profile (V2)" action**: After the trigger, add the **Get user profile (V2)** action.[1] In the **User ID** field, use the **From** field from the trigger to get the email address of the sender.[1]
4. **Extract the display name**: Configure the **Get user profile (V2)** action to retrieve the user profile information.[1] You can then use the **DisplayName** field from the output of this action to get the display name.[1]
5. **Store the display name in a variable**: Add a **Set variable** action to store the display name in a variable for later use.[1]

Here's an example of how the flow might look:

1. Trigger: When a new email arrives (V3)
2. Action: Get user profile (V2)
   - User ID: From (trigger)
3. Action: Set variable
   - Variable name: Display Name
   - Value: DisplayName (from Get user profile output)

This flow will automatically retrieve the display name of the requestor when a new email arrives and store it in a variable.

Does this help you achieve what you're looking for?

References

1. spguides.com

You

No so this is completely different to whoever owns the flow or runs it

Copilot

Got it. If you're looking to retrieve the display name of the requestor based on their email address, regardless of who owns or runs the flow, you can still use the **"Get user profile (V2)"** action in Power Automate. Here's how you can set it up to retrieve the requestor's details:

1. **Trigger your flow**: This could be any trigger that fits your use case, such as receiving an email, a new item being created in a SharePoint list, or a button press in Power Apps.
2. **Add the "Get user profile (V2)" action**: After the trigger, add the **Get user profile (V2)** action. In the **User (UPN)** field, use the email address of the requestor. For example, if you have the requestor's email in a variable or a field, use that as the input.

Here's a step-by-step example:

1. **Trigger**: Choose a trigger that fits your scenario. For example, **When a new item is created** in a SharePoint list.
2. **Get user profile (V2)**:
    - **User (UPN)**: Select the email address of the requestor. If you have the requestor's email from the trigger (e.g., a field in the SharePoint list), use that.
3. Get user profile (V2)
4. User (UPN): [Requestor's Email]
5. **Use the retrieved display name**: After retrieving the user profile, you can use the **DisplayName** field from the output in subsequent actions, such as sending an email or updating a record.

Here's an example of how the flow might look:

1. Trigger: When a new item is created (in SharePoint list)
2. Action: Get user profile (V2)
    - User (UPN): **dynamic content** (email of the requestor from the trigger)
3. Action: Use the **DisplayName** field from the **Get user profile (V2)** output as needed.

This way, you can retrieve the display name of the requestor based on their email address without depending on who owns or runs the flow.

Let me know if this helps or if you need further assistance!

You

(Now Tuesday, Feb 25)
What is a simple way in Power automate to get a file upload from a form and add it as an attachment to a SharePoint list

Copilot

To achieve this, you can create a Power Automate flow that captures the file upload from a Microsoft Form and adds it as an attachment to a SharePoint list item. Here's a step-by-step guide:

1. **Create a new flow**: Open Power Automate and create a new automated flow.
2. **Trigger**: Use the trigger **"When a new response is submitted"** from Microsoft Forms.
3. **Get response details**: Add the **"Get response details"** action to retrieve the details of the form submission. Select the form and use the response ID from the trigger.

4. **Parse JSON**: Add a **"Parse JSON"** action to parse the file upload details. Use the file upload question from the form as the content and provide the appropriate schema.
5. **Get file content**: Add a **"Get file content"** action to retrieve the file content from OneDrive. Use the file path from the parsed JSON.
6. **Create item in SharePoint**: Add a **"Create item"** action to create a new item in the SharePoint list. Fill in the necessary fields.
7. **Add attachment**: Add an **"Add attachment"** action to attach the file to the newly created SharePoint list item. Use the ID of the created item and the file content from the previous step.

Here's an example of how the flow might look:

1. **Trigger**: When a new response is submitted
2. **Action**: Get response details
    - Form ID: [Your Form ID]
    - Response ID: [Response ID from trigger]
3. **Action**: Parse JSON
    - Content: [File upload question from form]
    - Schema: [Appropriate JSON schema]
4. **Action**: Get file content
    - File path: [File path from parsed JSON]
5. **Action**: Create item
    - Site address: [Your SharePoint site address]
    - List name: [Your SharePoint list name]
    - Title: [Title from form response]
6. **Action**: Add attachment
    - Site address: [Your SharePoint site address]
    - List name: [Your SharePoint list name]
    - ID: [ID of created item]
    - File name: [File name from parsed JSON]
    - File content: [File content from Get file content action]

This flow will automatically add the uploaded file from the form as an attachment to the SharePoint list item.[1][2][3]

Does this help with what you're trying to achieve?

References

1. manueltgomes.com
1. spguides.com
1. techcommunity.microsoft.com

You

There is no schema for the Parse json, how else do I do it?

Copilot

No worries! If you don't have a predefined schema for the Parse JSON action, you can use an alternative method to handle file uploads from a form and add them as attachments to a SharePoint list. Here's how you can achieve this without the need for a Parse JSON action:

1. **Create a new flow**: Open Power Automate and create a new automated flow.
2. **Trigger**: Use the trigger **"When a new response is submitted"** from Microsoft Forms.
3. **Get response details**: Add the **"Get response details"** action to retrieve the details of the form submission. Select the form and use the response ID from the trigger.
4. **Extract file content**: Directly retrieve the file content without parsing JSON by using the dynamic content available from the form response.
5. **Create item in SharePoint**: Add a **"Create item"** action to create a new item in the SharePoint list. Fill in the necessary fields.
6. **Add attachment**: Add an **"Add attachment"** action to attach the file to the newly created SharePoint list item. Use the file content from the form response.

Here's an example of how the flow might look:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Action**: Get response details
    - Form ID: [Your Form ID]
    - Response ID: [Response ID from trigger]
3. **Action**: Create item
    - Site address: [Your SharePoint site address]
    - List name: [Your SharePoint list name]
    - Title: [Title from form response]
4. **Action**: Add attachment
    - Site address: [Your SharePoint site address]
    - List name: [Your SharePoint list name]
    - ID: [ID of created item]
    - File name: [File name from dynamic content]
    - File content: [File content from dynamic content]

This flow bypasses the need for parsing JSON and directly uses the dynamic content provided by the form response to retrieve and attach the file to the SharePoint list item.

Does this method work for your scenario? Let me know if you need further assistance!

You

What format is a file upload question received from the "Get response details" action?

Copilot

When you receive a file upload from a Microsoft Form in the "Get response details" action, the file upload question returns an array of file objects. Each file object includes details such as the file name, file content, and file type. Here is an example of the format you might receive:

```
[
  {
    "name": "example-file.png",
    "link": "https://your-tenant.sharepoint.com/sites/your-site/Documents/example-file.png"
  }
]
```

In this example, each object in the array represents a file, with two properties:

- name: The name of the file.
- link: The link to the file stored in OneDrive or SharePoint.

You can use the link property to download the file content.
To summarize, you don't need a JSON schema. Instead, use the link property directly from the file upload response. Here's how you can use it in your flow:

1. **Get response details**: Use the response ID from the trigger to get the form details.
2. **Loop through the file uploads**: Add an **"Apply to each"** action to loop through the array of file objects.
3. **Get file content**: Inside the loop, add the **"Get file content"** action and use the link property.
4. **Add attachment**: Add the file content as an attachment to the SharePoint list item.

Here is an example of the flow steps:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Apply to each**: Loop through the file upload array
    - **Get file content**: Use the link from the current item
4. **Create item**: Create a new item in the SharePoint list
5. **Add attachment**: Attach the file to the new item

I hope this helps! If you have any more questions, feel free to ask.

You

How do I get just the first element of the array and store it in a new variable?

Copilot

To get just the first element of the array and store it in a new variable in Power Automate, you can use the **"Compose"** action and the **"Initialize variable"** action. Here's a step-by-step guide on how to achieve this:

1. **Trigger your flow**: Choose the appropriate trigger for your flow, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Compose action to get the first element**: Add a **"Compose"** action to extract the first element of the array. Use an expression to get the first item.
4. **Initialize variable to store the first element**: Add an **"Initialize variable"** action to create a new variable and store the first element from the array.

Here's an example of how to set this up:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Compose**: Extract the first element of the array
   - Expression: first(<ArrayField>)
4. **Initialize variable**: Store the first element in a new variable
   - Name: FirstFile
   - Type: String
   - Value: outputs('Compose')

Here's how you can configure each action:

1. **Compose** action:
   - **Inputs**: Use the expression to get the first element:
     first(body('Get_response_details')?['<ArrayField>'])
   - Replace *<ArrayField>* with the name of your array field in the form response.
2. **Initialize variable** action:
   - **Name**: FirstFile
   - **Type**: String
   - **Value**: Use the dynamic content from the **"Compose"** action, which represents the first element of the array.

This setup will extract the first element of the array and store it in a new variable called FirstFile. You can then use this variable in subsequent actions within your flow.

Does this help with what you're trying to achieve? Let me know if you need any further assistance!

You

And how do I get the name and file contents from this FirstFile variable?

Copilot

To get the name and file contents from the FirstFile variable in Power Automate, you can extract the name and link properties from the JSON object. Here's how you can do this:

1. **Compose action to extract name and link**: Use the Compose action to extract the name and link properties.
2. **Get file content**: Use the Get file content using path action to retrieve the file contents using the link.

Here's a step-by-step guide:

1. **Trigger**: Use the trigger that fits your scenario.

2. **Get response details**: Use the response ID to get the form details.
3. **Compose to extract the first element**:
    - Action: Compose
    - Expression: *first(body('Get_response_details')?['<ArrayField>'])* Replace *<ArrayField>* with the name of your array field in the form response.
4. **Compose actions to extract name and link**:
    - Action: Compose (Name)
        - Expression: *outputs('Compose')?['name']*
    - Action: Compose (Link)
        - Expression: *outputs('Compose')?['link']*
5. **Get file content**:
    - Action: Get file content using path
    - File path: Use the dynamic content from the Compose (Link) action
6. **Add attachment**:
    - Site address: [Your SharePoint site address]
    - List name: [Your SharePoint list name]
    - ID: [ID of created item]
    - File name: Use the dynamic content from the Compose (Name) action
    - File content: Use the dynamic content from the Get file content action

Here's how you can configure each action:

1. **Compose (First Element)**:
    - *first(body('Get_response_details')?['<ArrayField>'])*

   Replace *<ArrayField>* with the name of your array field in the form response.
2. **Compose (Name)**:
    - *outputs('Compose')?['name']*
3. **Compose (Link)**:
    - *outputs('Compose')?['link']*
4. **Get file content using path**:
    - **File path**: Use the dynamic content from the Compose (Link) action

This setup will extract the name and link of the first file from the array and retrieve the file content. You can then use this information to add the file as an attachment to a SharePoint list item.

Does this help you achieve what you're looking for? Let me know if you need further assistance!

You

What variable type does my first compose return?

Copilot

The output of your first Compose action, which extracts the first element of the array, will be a JSON object. This JSON object contains the properties of the file (e.g., name and link).

In Power Automate, JSON objects are handled as string data types. However, you can still reference and extract specific properties from the JSON object using expressions.

For example, if your Compose action is named Compose, the expressions to extract the name and link properties are:

- Name: *outputs('Compose')?['name']*
- Link: *outputs('Compose')?['link']*

If you need to store these properties in separate variables, you can use the **"Initialize variable"** action and set the variable value using the above expressions.

Here's a quick example of how to initialize and set these variables:

1. **Initialize variable for file name**:
    - Name: *FileName*
    - Type: *String*
    - Value: *outputs('Compose')?['name']*
2. **Initialize variable for file link**:
    - Name: *FileLink*
    - Type: *String*
    - Value: *outputs('Compose')?['link']*

These actions will store the name and link properties from the first element of your JSON array into separate string variables, which you can use in subsequent actions.

Does this help clarify things? If you need further assistance, feel free to ask!

You

I got the following error when trying to initialize the variable:
Unable to process template language expressions in action 'Initialize_variable' inputs at line '0' and column '0': 'The template language expression 'outputs('Compose')['name']' cannot be evaluated because property 'name' cannot be selected. Property selection is not supported on values of type 'String'. Please see https://aka.ms/logicexpressions for usage details.'.

Copilot

The issue you're encountering is because the Compose action returns a string, and you cannot directly access the properties of a JSON object in the string. To resolve this, you can use the Parse JSON action after the Compose action to properly handle the JSON object.

Here's how you can update your flow:

1. **Get response details**: Retrieve the details of the form submission.
2. **Compose action to extract the first element**: Extract the first element of the array.
3. **Parse JSON action**: Parse the JSON object from the Compose action.
4. **Initialize variables**: Extract the name and link properties and store them in variables.

Here's a step-by-step guide:

1. **Compose (First Element)**:
    - **Inputs**: Use the expression to get the first element:
      first(body('Get_response_details')?['<ArrayField>'])
    - Replace *<ArrayField>* with the name of your array field in the form response.
2. **Parse JSON** action:
    - **Content**: Use the output of the Compose action.
    - **Schema**: Use the following schema to parse the JSON:

```
{
  "type": "object",
  "properties": {
    "name": {
      "type": "string"
    },
    "link": {
      "type": "string"
    }
  }
}
```

3. **Initialize variable for file name**:
    - **Name**: FileName
    - **Type**: String
    - **Value**: Use the dynamic content name from the Parse JSON action.
4. **Initialize variable for file link**:
    - **Name**: FileLink
    - **Type**: String

    - **Value**: Use the dynamic content link from the Parse JSON action.

By parsing the JSON object, you can then extract the properties name and link and store them in variables. Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Compose (First Element)**: Extract the first element of the array
    - Expression: *first(body('Get_response_details')?['<ArrayField>'])*
4. **Parse JSON**:
    - Content: Use the output of the Compose action
    - Schema: Use the provided JSON schema
5. **Initialize variable for file name**:
    - Name: FileName
    - Type: String
    - Value: Dynamic content name from Parse JSON
6. **Initialize variable for file link**:
    - Name: FileLink
    - Type: String
    - Value: Dynamic content link from Parse JSON

This will allow you to correctly handle and store the file upload properties. Let me know if you need any further assistance!

You

InvalidJSON
The 'content' property of actions of type 'ParseJson' must be valid JSON. The provided value cannot be parsed: 'Unexpected end when reading token. Path ".'.

Copilot

To resolve the "Invalid JSON" error and correctly parse the file upload details from the form, you can use a different approach to handle the JSON content. Instead of directly parsing the JSON, you can use expressions to access the properties of the first element in the array.

Here's a step-by-step guide to achieve this:

1. **Trigger**: Use the trigger that fits your scenario, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Compose action to extract the first element**: Add a **"Compose"** action to extract the first element of the array.
4. **Initialize variable for file name**: Add an **"Initialize variable"** action to create a new variable and store the file name.
5. **Initialize variable for file link**: Add an **"Initialize variable"** action to create a new variable and store the file link.

Here's how you can set this up:

1. **Compose (First Element)**:
    - **Inputs**: Use the expression to get the first element:
      first(body('Get_response_details')?['<ArrayField>'])
    - Replace *<ArrayField>* with the name of your array field in the form response.
2. **Initialize variable for file name**:
    - **Name**: FileName
    - **Type**: String
    - **Value**: Use the expression:
      first(body('Get_response_details')?['<ArrayField>'])['name']
3. **Initialize variable for file link**:
    - **Name**: FileLink
    - **Type**: String

- **Value**: Use the expression:
  first(body('Get_response_details')?['<ArrayField>'])['link']

Here's how the flow steps might look:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Compose (First Element)**: Extract the first element of the array
   Expression: first(body('Get_response_details')?['<ArrayField>'])
4. **Initialize variable for file name**:
   - Name: FileName
   - Type: String
   - Value: Expression:
     first(body('Get_response_details')?['<ArrayField>'])['name']
5. **Initialize variable for file link**:
   - Name: FileLink
   - Type: String
   - Value: Expression:
     first(body('Get_response_details')?['<ArrayField>'])['link']

This setup will allow you to directly access and store the file name and link properties without encountering JSON parsing errors.

Give this approach a try and let me know if it resolves the issue!

You

InvalidTemplate
Unable to process template language expressions in action 'Initialize_variable' inputs at line '0' and column '0': 'The template language expression 'first(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'])['name']' cannot be evaluated because property 'name' cannot be selected. Property selection is not supported on values of type 'String'. Please see https://aka.ms/logicexpressions for usage details.'.

Copilot

It looks like the error occurs because the output of the Compose action is treated as a string, so you cannot directly access its properties. Instead, you can use the Select action to properly handle the file details and extract the required properties. Here's an alternative approach:

1. **Trigger**: Use the trigger that fits your scenario, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Apply to each**: Use the Apply to each action to loop through the array of file objects.
4. **Select**: Use the Select action to map the file properties to a new JSON object.
5. **Compose**: Use the Compose action to extract the first element of the mapped JSON array.
6. **Initialize variables**: Extract the name and link properties and store them in variables.

Here's how you can set this up:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Apply to each**:
   - **From**: The file upload array (dynamic content from form response)
4. **Select**:
   - **Map**: Create a new JSON object with name and link

```
[
  {
    "name": item()['name'],
    "link": item()['link']
  }
]
```

5. **Compose (First Element)**:
    - **Inputs**: Use the expression to get the first element:
        first(outputs('Select'))
6. **Initialize variable for file name**:
    - **Name**: FileName
    - **Type**: String
    - **Value**: Use the expression:
        outputs('Compose')['name']
7. **Initialize variable for file link**:
    - **Name**: FileLink
    - **Type**: String
    - **Value**: Use the expression:
        outputs('Compose')['link']

Here's the flow structure:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Apply to each**: Loop through the file upload array
    - **Select**: Map the file properties to a new JSON object
4. **Compose (First Element)**: Extract the first element of the mapped JSON array
    - Expression: *first(outputs('Select'))*
5. **Initialize variable for file name**:
    - Name: FileName
    - Type: String
    - Value: Expression: *outputs('Compose')['name']*
6. **Initialize variable for file link**:
    - Name: FileLink
    - Type: String
    - Value: Expression: *outputs('Compose')['link']*

By mapping the file properties with the Select action, you can properly handle the JSON object and extract the required properties without encountering errors.

Give this approach a try and let me know if it resolves the issue!

You

This is the output of my file upload object from the form:
[{"name":"5175840487_Akhtar Hasan.pdf","link":"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/Documents/Don%27t%20need%20backup/Apps/Microsoft%20Forms/Request%20form%20for%20SAP/Question/5175840487_Akhtar%20Hasan.pdf","id":"01ZBAD6XCXJWFVRQQXFJEYHDXWNJX5TXEN","type":null,"size":91033,"referenceId":"01ZBAD6XFVCUXPHDA7UVCYFLJ2YOJBTKNH","driveId":"b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0","status":1,"uploadSessionUrl":null}]
It should be an array, but it's not detected as an array at all

Copilot

Thanks for sharing the output! Since the file upload object is indeed an array, let's take an alternative approach to handle it correctly. We can use the Select action to map the properties of the file and then use the Compose action to extract the first element. Here's how you can do it step-by-step:

1. **Trigger your flow**: Use the appropriate trigger, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file array**:
    - **Name**: FileArray
    - **Type**: Array
    - **Value**: Use the dynamic content from the file upload question.

4. **Apply to each**: Use the **Apply to each** action to loop through the file array.
    - **From**: Use the **FileArray** variable.
5. **Select**: Use the **Select** action to map the file properties to a new JSON object.
    - **Map**: Create a new JSON object with name and link:

```
{
  "name": item()?['name'],
  "link": item()?['link']
}
```

6. **Compose (First Element)**: Use the **Compose** action to extract the first element of the mapped JSON array.
    - **Inputs**: Use the expression to get the first element:
      first(outputs('Select'))
7. **Initialize variable for file name**:
    - **Name**: FileName
    - **Type**: String
    - **Value**: Use the expression:
      outputs('Compose')['name']
8. **Initialize variable for file link**:
    - **Name**: FileLink
    - **Type**: String
    - **Value**: Use the expression:
      outputs('Compose')['link']

Here's the flow structure:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
    - **Name**: FileArray
    - **Type**: Array
    - **Value**: Use the dynamic content from the file upload question
4. **Apply to each**: Loop through the file array
    - **From**: Use the **FileArray** variable
5. **Select**: Map the file properties to a new JSON object
    - **Map**: Create a new JSON object with name and link
6. **Compose (First Element)**: Extract the first element of the mapped JSON array
    - **Inputs**: Use the expression *first(outputs('Select'))*
7. **Initialize variable for file name**:
    - **Name**: FileName
    - **Type**: String
    - **Value**: Expression: *outputs('Compose')['name']*
8. **Initialize variable for file link**:
    - **Name**: FileLink
    - **Type**: String
    - **Value**: Expression: *outputs('Compose')['link']*

This approach should correctly handle the file array and allow you to extract the name and link properties without encountering errors. Give this a try and let me know if it works for you!

You

No variables come up when I use select, how can I extract the information if the full thing is treated as a string:
[{"name":"5175840487_Akhtar Hasan.pdf","link":"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/Documents/Don%27t%20need%20backup/Apps/Microsoft%20Forms/Request%20form%20for%20SAP/Question/5175840487_Akhtar%20Hasan.pdf","id":"01ZBAD6XCXJWFVRQQXFJEYHDXWNJX5TXEN","type":null,"size":91033,"referenceId":"01ZBAD6XFVCUXPHDA7UVCYFLJ2YOJBTKNH","driveId":"b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0","status":1,"uploadSessionUrl":null}]

Copilot

Since the file upload object from the form is treated as a string, you can use the **"Parse JSON"** action to convert the string into a JSON object that can be easily handled. Here's how you can do it:

1. **Trigger**: Use the trigger that fits your scenario, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file array**:
   - **Name**: FileArray
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
4. **Parse JSON**: Use the **"Parse JSON"** action to convert the string into a JSON object.
   - **Content**: Use the FileArray variable.
   - **Schema**: Use the following schema:

```
{
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "name": {
        "type": "string"
      },
      "link": {
        "type": "string"
      }
    }
  }
}
```

5. **Compose (First Element)**: Use the **Compose** action to extract the first element of the JSON array.
   - **Inputs**: Use the expression to get the first element:
     first(outputs('Parse_JSON'))
6. **Initialize variable for file name**:
   - **Name**: FileName
   - **Type**: String
   - **Value**: Use the expression:
     outputs('Compose')['name']
7. **Initialize variable for file link**:
   - **Name**: FileLink
   - **Type**: String
   - **Value**: Use the expression:
     outputs('Compose')['link']

Here's the flow structure:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileArray
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Parse JSON**:
   - **Content**: Use the FileArray variable
   - **Schema**: Use the provided JSON schema
5. **Compose (First Element)**: Extract the first element of the JSON array
   - **Inputs**: Use the expression first(outputs('Parse_JSON'))
6. **Initialize variable for file name**:
   - **Name**: FileName
   - **Type**: String
   - **Value**: Expression: outputs('Compose')['name']
7. **Initialize variable for file link**:
   - **Name**: FileLink
   - **Type**: String
   - **Value**: Expression: outputs('Compose')['link']

This approach should correctly handle the file array and allow you to extract the name and link properties without encountering errors. Give this a try and let me know if it works for you!

You

How can I use string methods to get the file name and file link separately?
What is the flow action that I can use to upload the file that has been uploaded in the form to a SharePoint site?
So the path from my file upload is a personal OneDrive path e.g. "path": "https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B01D988B9-90D5-4F9E-8AAF-12E56C7FDEB5%7D&file=Case%201%20(4)_Akhtar%20Hasan.docx&action=default&mobileredirect=true", how can I get the file contents from this?

Copilot

To get the file contents from a personal OneDrive path like the one you provided, you'll need to use the **"Get file content"** action in Power Automate. This action retrieves the file content using a file path. However, you'll need to make sure the file path is formatted correctly and accessible.

Here's how you can set it up:

1. **Trigger your flow**: Use the appropriate trigger, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file path**:
    - **Name**: FilePath
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question (the link to the file).
4. **Convert the file path**: Extract the relevant part of the URL to use as the file path in the OneDrive connector.
5. **Get file content**: Use the **"Get file content using path"** action to retrieve the file contents from OneDrive.
    - **File path**: Use the extracted file path.

Here's a step-by-step guide:

1. **Trigger**: Use the trigger that fits your scenario.
2. **Get response details**: Use the response ID to get the form details.
3. **Initialize a variable to store the file path**:
    - **Name**: FilePath
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question.
4. **Compose (Extracted Path)**: Add a **Compose** action to extract the relevant part of the URL:
    - **Expression**: Use the following expression to extract the path:
      decodeUriComponent(first(split(outputs('Initialize_variable'),'?'))['path'])
5. **Get file content using path**:
    - **File path**: Use the output from the **Compose (Extracted Path)** action.

Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
    - **Name**: FilePath
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Path)**: Extract the relevant part of the URL
    - **Expression**:
      decodeUriComponent(first(split(outputs('Initialize_variable'),'?'))['path'])
5. **Get file content using path**:
    - **File path**: Use the output from the **Compose (Extracted Path)** action

This setup will allow you to retrieve the file contents from the personal OneDrive path provided in the form response.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

What would be the 'relevant part' in a url such as this:
"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B01D988B9-90D5-4F9E-8AAF-12E56C7FDEB5%7D&file=Case%201%20(4)_Akhtar%20Hasan.docx&action=default&mobileredirect=true"

Copilot

The relevant part of the URL you provided that points to the file path in OneDrive would typically be the part after the file= parameter. This is the direct path to the file itself.

In the URL you provided:

"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B01D988B9-90D5-4F9E-8AAF-12E56C7FDEB5%7D&file=Case%201%20(4)_Akhtar%20Hasan.docx&action=default&mobileredirect=true"

The relevant part is:

Case%201%20(4)_Akhtar%20Hasan.docx

So, when constructing the file path for the Get file content action, you should decode and extract this portion. Here's how you can do it:

1. **Initialize a variable to store the file path**:
   - **Name**: FilePath
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
2. **Compose (Extracted Path)**: Add a **Compose** action to extract the relevant part of the URL:
   - **Expression**: Use the following expression to extract the part after file=:
     decodeUriComponent(last(split(triggerOutputs()?['body/your_file_upload_field'], 'file=')))
     Replace your_file_upload_field with the actual field name that contains the file URL.
3. **Get file content using path**:
   - **File path**: Use the output from the **Compose (Extracted Path)** action.

Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FilePath
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Path)**: Extract the relevant part of the URL
   - **Expression**:
     decodeUriComponent(last(split(triggerOutputs()?['body/your_file_upload_field'], 'file=')))
5. **Get file content using path**:
   - **File path**: Use the output from the **Compose (Extracted Path)** action

This will help you extract the relevant part of the URL and get the file content from the personal OneDrive path.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

This is what has been extracted by the compose:
Case 1 (4)_Akhtar Hasan.docx&action=default&mobileredirect=true

Copilot

It looks like the extraction grabbed more than just the file name. Let's refine the extraction to isolate the file name part and exclude the additional query parameters.

Here's a refined approach:

1. **Compose (Extracted Path)**: Add a **Compose** action to extract the relevant part of the URL:
    - **Expression**: Use the following expression to extract only the file name:
2. decodeUriComponent(first(split(last(split(outputs('Initialize_variable'), 'file=')), '&')))

This expression does the following:

1. Splits the URL at the file= parameter and takes the last part.
2. Splits the result at the & character to remove any query parameters and takes the first part.
3. Decodes the URL component to get the actual file name.

Here's what the updated flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
    - **Name**: FilePath
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Path)**: Extract the relevant part of the URL
    - **Expression**:
5. decodeUriComponent(first(split(last(split(outputs('Initialize_variable'), 'file=')), '&')))

This should result in Case 1 (4)_Akhtar Hasan.docx, excluding the query parameters.

5. **Get file content using path**:
    - **File path**: Use the output from the **Compose (Extracted Path)** action

Give this refined approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

Now the path is:
https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/Documents/Don't need backup/Apps/Microsoft Forms/Request form for SAP/Question/5175840487_Akhtar Hasan.pdf

Copilot

Great! Now that you have the correct path to the file, you can use the **"Get file content using path"** action to retrieve the file content from OneDrive. Here's how you can set it up:

1. **Trigger**: Use the trigger that fits your scenario, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file path**:
    - **Name**: FilePath
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question (the link to the file).
4. **Compose (Extracted Path)**: Add a **Compose** action to extract the relevant part of the URL:
    - **Expression**: Use the following expression to extract the part after Documents/:
        decodeUriComponent(replace(first(split(outputs('FilePath'), 'Documents/')), '%20', ' '))
5. **Get file content using path**:
    - **File path**: Combine the OneDrive site base path with the extracted file path.

Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
    - **Name**: FilePath
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Path)**: Extract the relevant part of the URL
    - **Expression**:
        decodeUriComponent(replace(first(split(outputs('FilePath'), 'Documents/')), '%20', ' '))
5. **Get file content using path**:
    - **File path**: Combine the OneDrive site base path with the output from the **Compose (Extracted Path)** action
    - */personal/akhtar_hasan_bradfordcft_org_uk/Documents/*

Using this approach, you should be able to retrieve the file content from the personal OneDrive path. Give this a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

Even when the path is:
Case 1 (4)_Akhtar Hasan.docx
I still get the following error:
Action 'Get_file_content_using_path_1' failed: The resource could not be found.

Copilot

It sounds like there might be an issue with the exact file path used in the **"Get file content using path"** action. Let's try a different approach to ensure we are correctly accessing the file in OneDrive.

1. **Trigger**: Use the appropriate trigger, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file URL**:
    - **Name**: FileURL
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question.
4. **Compose (Extract Drive ID and Item ID)**: Add a **Compose** action to extract the driveId and id from the file URL:
    - **Expression**: Use the following expression to extract the relevant parts:

```
{
   "driveId": "<driveId>",
   "id": "<itemId>"
}
```

Replace <driveId> and <itemId> with the actual values from the file URL.

5. **Get file content**: Use the **"Get file content"** action to retrieve the file contents from OneDrive using the driveId and itemId.

Here's how the flow steps might look:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extract Drive ID and Item ID)**:
   - **Expression**:
     ```
     {
       "driveId": "<driveId>",
       "id": "<itemId>"
     }
     ```
5. **Get file content**:
   - **Drive ID**: Use the extracted *driveId*
   - **ID**: Use the extracted *itemId*

By extracting the *driveId* and *itemId* from the file URL, you can use these values directly in the **"Get file content"** action to correctly access the file in OneDrive.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B01D988B9-90D5-4F9E-8AAF-12E56C7FDEB5%7D&file=Case%201%20(4)_Akhtar%20Hasan.docx&action=default&mobileredirect=true"
How can I get the unique file identifier from a string such as the above

Copilot

To extract the unique file identifier from the URL, you can parse the value of the sourcedoc parameter. Here's a step-by-step approach:

1. **Extract the parameter value from the URL**: Use string operations to extract the value of the sourcedoc parameter from the given URL.

You can use the **Compose** action with an expression to achieve this. Here's how you can do it:

1. **Compose (Extracted Unique Identifier)**:
   - **Expression**: Use the following expression to extract the value of the sourcedoc parameter:
     first(split(split('<Your URL>', 'sourcedoc=')?[-1], '&'))

Replace <Your URL> with the actual URL string from the file upload.

Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Unique Identifier)**:
   - **Expression**:
     first(split(split(outputs('Initialize_variable'), 'sourcedoc=')?[-1], '&'))

This expression does the following:

1. Splits the URL at the sourcedoc= parameter and takes the last part.
2. Splits the result at the & character to remove any additional query parameters and takes the first part.

The result should give you the unique file identifier: {*01D988B9-90D5-4F9E-8AAF-12E56C7FDEB5*}.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

first(split(split(outputs('Initialize_variable'), 'sourcedoc=')?[-1], '&'))
I keep getting this error with the above syntax:
Action 'Compose' failed: Unable to process template language expressions in action 'Compose' inputs at line '0' and column '0': 'The template language function 'split' expects its first parameter to be of type string. The provided value is of type 'Null'. Please see https://aka.ms/logicexpressions#split for usage details.'.

Copilot

The error occurs because the value being split is null. To ensure we're working with a valid string, let's first confirm the FileURL variable contains the URL. Additionally, let's add a check to ensure the URL contains the sourcedoc parameter before attempting to split it.

Here's the refined approach:

1. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
2. **Compose (Validate URL)**: Add a **Compose** action to validate the URL and ensure it contains the sourcedoc parameter:
   - **Expression**:
     if(equals(indexOf(outputs('Initialize_variable'), 'sourcedoc='), -1), 'Invalid URL', outputs('Initialize_variable'))
3. **Compose (Extracted Unique Identifier)**: Add a **Compose** action to extract the value of the sourcedoc parameter if the URL is valid:
   - **Expression**:
     if(equals(outputs('Compose_Validate_URL'), 'Invalid URL'), 'No sourcedoc parameter found', first(split(split(outputs('Compose_Validate_URL'), 'sourcedoc=')[-1], '&')))

Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Validate URL)**: Validate the URL and ensure it contains the sourcedoc parameter
   - **Expression**:
     if(equals(indexOf(outputs('Initialize_variable'), 'sourcedoc='), -1), 'Invalid URL', outputs('Initialize_variable'))
5. **Compose (Extracted Unique Identifier)**: Extract the value of the sourcedoc parameter if the URL is valid
   - **Expression**:
     if(equals(outputs('Compose_Validate_URL'), 'Invalid URL'), 'No sourcedoc parameter found', first(split(split(outputs('Compose_Validate_URL'), 'sourcedoc=')[-1], '&')))

This refined approach ensures we are only splitting valid URLs and properly handling cases where the sourcedoc parameter might be missing.

Give this refined approach a try and let me know if it resolves the issue! If you need further assistance, feel free to ask.

You

Unable to process template language expressions in action 'Compose' inputs at line '0' and column '0': 'The template language function 'split' expects its first parameter to be of type string. The provided value is of type 'Null'. Please see https://aka.ms/logicexpressions#split for usage details.'.

Copilot

The error suggests that the split function received a null value instead of a string. Let's add a check to ensure that the value isn't null before performing the split. Here's how you can update the flow to handle this:

1. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
2. **Compose (Extracted Unique Identifier)**: Add a **Compose** action to extract the value of the sourcedoc parameter:
   - **Expression**: Use the following expression to handle the case where the sourcedoc parameter might be null:

     if(equals(triggerOutputs()?['body/your_file_upload_field'], null), 'No URL provided', first(split(split(triggerOutputs()?['body/your_file_upload_field'], 'sourcedoc=')?[-1], '&')))

Replace your_file_upload_field with the actual field name that contains the file URL.

Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Unique Identifier)**: Extract the value of the sourcedoc parameter
   - **Expression**:
   - if(equals(triggerOutputs()?['body/your_file_upload_field'], null), 'No URL provided', first(split(split(triggerOutputs()?['body/your_file_upload_field'], 'sourcedoc=')?[-1], '&')))

This expression ensures that we handle cases where the URL might be null and properly extract the *sourcedoc* parameter if it exists.

Give this refined approach a try and let me know if it resolves the issue! If you need further assistance, feel free to ask.

You

Now it works, but the file is never actually attached, whereas the process that I'm trying to build requires the file to be attached to the SharePoint list item

Copilot

Let's make sure we have all the necessary steps to attach the file to the SharePoint list item. Here's a step-by-step guide to ensure the file is correctly attached:

1. **Trigger**: Use the trigger that fits your scenario, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.

4. **Compose (Extracted Unique Identifier)**: Add a **Compose** action to extract the value of the sourcedoc parameter:
    - **Expression**:
      first(split(split(triggerOutputs()?['body/your_file_upload_field'], 'sourcedoc=')?[-1], '&'))
5. **Get file content using ID**:
    - **Action**: **Get file content using path**
    - **File path**: Use the extracted file path
6. **Create item in SharePoint**: Add a **"Create item"** action to create a new item in the SharePoint list. Fill in the necessary fields.
7. **Add attachment to SharePoint list item**:
    - **Action**: **Add attachment**
    - **Site address**: [Your SharePoint site address]
    - **List name**: [Your SharePoint list name]
    - **ID**: Use the ID of the newly created item
    - **File name**: Use the extracted file name
    - **File content**: Use the dynamic content from the **Get file content** action

Here's how the full flow might look:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
    - **Name**: FileURL
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Unique Identifier)**: Extract the value of the sourcedoc parameter
    - **Expression**:
      first(split(split(triggerOutputs()?['body/your_file_upload_field'], 'sourcedoc=')?[-1], '&'))
5. **Get file content using path**:
    - **File path**: Use the extracted file path
6. **Create item in SharePoint**:
    - **Site address**: Your SharePoint site address
    - **List name**: Your SharePoint list name
    - **Title**: Use the form response data (as needed)
7. **Add attachment**:
    - **Site address**: Your SharePoint site address
    - **List name**: Your SharePoint list name
    - **ID**: Use the ID of the newly created item
    - **File name**: Use the extracted file name
    - **File content**: Use the dynamic content from the **Get file content using path** action

This should correctly attach the uploaded file to the newly created SharePoint list item. Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

How can I change this substring method to extract only the parts around '*sourcedoc*' instead?

```
substring(
  outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'],
  add(indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], '"link":"'), 8),
  sub(
    indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], '","id":"'),
    add(indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], '"link":"'), 8)
  )
)
```

Copilot

To extract only the part around '*sourcedoc*', you can modify the substring method to focus on the *sourcedoc* parameter in the URL. The goal is to get the value of the *sourcedoc* parameter between *sourcedoc*= and the next *&* character.

Here's how you can do it:

1. **Locate the starting position of the *sourcedoc*= parameter**.
2. **Find the position of the next & character** after *sourcedoc*=.
3. **Extract the substring between these two positions**.

Here is the modified version of your substring method:

```
substring(
   outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'],
   add(indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], 'sourcedoc='), 10),
   sub(
      indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], '&',
add(indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], 'sourcedoc='), 10)),
      add(indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], 'sourcedoc='), 10)
   )
)
```

Explanation:

1. indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], 'sourcedoc=') finds the starting position of the sourcedoc= parameter.
2. add(indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], 'sourcedoc='), 10) adds 10 to this position to get to the end of sourcedoc= and the start of the unique identifier.
3. indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], '&', add(indexOf(outputs('Get_response_details')?['body/rd724feb2050a4215983b20be560e4042'], 'sourcedoc='), 10)) finds the position of the next & character after sourcedoc=.
4. The substring method then extracts the part between these two positions.

Give this a try and see if it extracts the unique file identifier correctly. If you need any more help, feel free to ask!

You

Action 'Initialize_variable_1' failed: Unable to process template language expressions in action 'Initialize_variable_1' inputs at line '0' and column '0': 'The template language function 'indexOf' expects two parameters: the string as the first parameter, the value to find the index of as the second parameter. The function was invoked with '3' parameter(s). Please see https://aka.ms/logicexpressions#indexof for usage details.'.
I have this as my Compose 1:
if(equals(indexOf(variables('fileID'), 'sourcedoc='), -1), 'Invalid URL', variables('fileID'))
However variables('fileID') is now %7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D
How can I modify the formula in Compose 1?

Copilot

The %7B and %7D sequences are URL-encoded representations of { and } respectively. To decode these, you can modify the formula to handle the URL-encoded string properly. Here's how you can decode the URL before performing further operations:

1. **Compose (Decode URL)**: Add a **Compose** action to decode the URL:
   - **Expression**:
     ```
     decodeUriComponent(variables('fileID'))
     ```
2. **Compose (Extracted Unique Identifier)**: Use the decoded URL and modify your formula to extract the value of the sourcedoc parameter:
   - **Expression**:
     ```
     if(equals(indexOf(outputs('Compose_Decode_URL'), 'sourcedoc='), -1), 'Invalid URL',
     first(split(split(outputs('Compose_Decode_URL'), 'sourcedoc=')?[-1], '&')))
     ```

Here's how the flow steps might look:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
    - **Name**: fileID
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question
4. **Compose (Decode URL)**:
    - **Expression**:
        decodeUriComponent(variables('fileID'))
5. **Compose (Extracted Unique Identifier)**:
    - **Expression**:
        if(equals(indexOf(outputs('Compose_Decode_URL'), 'sourcedoc='), -1), 'Invalid URL', first(split(split(outputs('Compose_Decode_URL'), 'sourcedoc=')?[-1], '&')))

This approach decodes the URL first, then performs the substring operations to extract the sourcedoc parameter value. Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

How do I remove the "*%7B*" and "*%7D*" from *variables('fileID')*?

Copilot

To remove the %7B and %7D from variables('fileID'), you can use the replace function in Power Automate. This function allows you to replace specific substrings with an empty string, effectively removing them. Here's how you can do it:

1. **Initialize a variable to store the file ID**:
    - **Name**: fileID
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question.
2. **Compose (Remove %7B and %7D)**: Add a **Compose** action to remove the %7B and %7D substrings from variables('fileID'):
    - **Expression**:
        replace(replace(variables('fileID'), '%7B', ''), '%7D', '')

This expression performs the following:

1. Replaces %7B with an empty string.
2. Then, replaces %7D with an empty string.

Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
    - **Name**: fileID
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question
4. **Compose (Remove %7B and %7D)**:
    - **Expression**:
        replace(replace(variables('fileID'), '%7B', ''), '%7D', '')

This will remove the *%7B* and *%7D* substrings from *variables('fileID')*, leaving you with the cleaned-up identifier.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

My full file url was:
https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D&file=New%20A%20%26%20E%20Case%201%20(1)_Akhtar%20Hasan.docx&action=default&mobileredirect=true
However when I use the 'Get File content' OneDrive action I get the following error: Action 'Get_file_content' failed: The file id '*79406FD8-7571-41EE-AAE8-1AA56C9903B0'* is not valid.

Copilot

It looks like we're trying to use the file ID from the SharePoint link incorrectly. Given the structure of the URL and the unique identifier we extracted, we need a different method to access the file content. The file ID from the URL (79406FD8-7571-41EE-AAE8-1AA56C7FDEB5) is likely part of the document library metadata rather than the actual file.

Here's an alternative method to retrieve the file content using the link provided by the form submission:

1. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
2. **Get file content using the link**:
   - **Action**: **Get file content using path**
   - **File path**: Extract and use the relevant part of the file URL to create the proper file path.

Here's the revised step-by-step guide:

1. **Trigger**: Use the trigger that fits your scenario, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
4. **Compose (Extracted Path)**: Add a **Compose** action to extract the file path from the URL:
   - **Expression**:
     replace(substring(triggerOutputs()?['body/your_file_upload_field'], indexOf(triggerOutputs()?['body/your_file_upload_field'], '/personal/')), -1, -1)
5. **Get file content using path**:
   - **File path**: Use the output from the **Compose (Extracted Path)** action.

Here's how the updated flow might look:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Path)**: Extract the relevant part of the URL
   - **Expression**:
     replace(substring(triggerOutputs()?['body/your_file_upload_field'], indexOf(triggerOutputs()?['body/your_file_upload_field'], '/personal/')), -1, -1)
5. **Get file content using path**:
   - **File path**: Use the output from the **Compose (Extracted Path)** action

By extracting the relevant file path from the URL, you should be able to use the Get file content using path action to correctly retrieve the file content.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

Action 'Get_file_content_using_path' failed: Unable to process template language expressions in action 'Get_file_content_using_path' inputs at line '0' and column '0': 'The template language function 'replace' expects its second parameter 'old string' to be a string. The provided value is of type 'Integer'. Please see https://aka.ms/logicexpressions#replace for usage details.'.

Copilot

The error you're encountering happens because the replace function requires a string for the old value to be replaced. Let's refine the approach to make sure we're getting the correct file path from the URL.

Here's the revised approach:

1. **Extract the file path**: Use string functions to isolate the relevant part of the URL.
2. **Get file content using path**: Use the extracted file path to get the file content.

Let's break it down step-by-step:

1. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
2. **Compose (Extract File Path)**: Add a **Compose** action to extract the file path from the URL:
   - **Expression**:

```
substring(
  outputs('Initialize_variable')?['body/your_file_upload_field'],
  indexOf(outputs('Initialize_variable')?['body/your_file_upload_field'], '/personal/'),
  sub(
    length(outputs('Initialize_variable')?['body/your_file_upload_field']),
    indexOf(outputs('Initialize_variable')?['body/your_file_upload_field'], '/personal/')
  )
)
```

3. **Get file content using path**:
   - **File path**: Use the output from the **Compose (Extract File Path)** action.

Here's how the updated flow might look:

1. **Trigger**: When a new response is submitted (Microsoft Forms).
2. **Get response details**: Use the response ID.
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
4. **Compose (Extract File Path)**:
   - **Expression**:

```
substring(
  outputs('Initialize_variable')?['body/your_file_upload_field'],
  indexOf(outputs('Initialize_variable')?['body/your_file_upload_field'], '/personal/'),
  sub(
    length(outputs('Initialize_variable')?['body/your_file_upload_field']),
    indexOf(outputs('Initialize_variable')?['body/your_file_upload_field'], '/personal/')
  )
)
```

5. **Get file content using path**:
   - **File path**: Use the output from the **Compose (Extract File Path)** action.

This setup should help you extract the relevant file path from the URL and retrieve the file content without encountering the error.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

This is the path:
"/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D&file=New%20A%20%26%20E%20Case%201%20(1)_Akhtar%20Hasan.docx&action=default&mobileredirect=true","id":"01ZBAD6XGYN5AHS4LV5ZA2V2A2UVWJSA5Q","type":null,"size":26318,"referenceId":"01ZBAD6XFVCUXPHDA7UVCYFLJ2YOJBTKNH","driveId":"b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0"
However I get this error:
Action 'Get_file_content_using_path' failed: A potentially dangerous Request.Path value was detected from the client (?).

Copilot

The error you are encountering is related to the URL encoding and the presence of potentially unsafe characters in the path. Since the path contains encoded characters and other parameters, it is causing issues when trying to use it in the Get file content using path action.

Let's break down and refine our approach to properly extract the file content using the unique driveId and id parameters extracted from the file URL, as this can provide a more reliable method to access the file.
Here's how you can proceed:

1. **Extract the driveId and id from the URL**: Use string functions to extract these parameters from the file URL.
2. **Get file content using driveId and id**: Use the Get file content action with these parameters to get the file content.

Here's a step-by-step guide:

1. **Trigger your flow**: Use the appropriate trigger, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
4. **Compose (Extract driveId)**: Add a **Compose** action to extract the driveId from the URL:
   - **Expression**:
     first(split(last(split(triggerOutputs()?['body/your_file_upload_field'], 'driveId=')), ''))
5. **Compose (Extract id)**: Add a **Compose** action to extract the id from the URL:
   - **Expression**:
     first(split(last(split(triggerOutputs()?['body/your_file_upload_field'], 'id=')), ''))
6. **Get file content using driveId and id**:
   - **Action**: **Get file content**
   - **Drive ID**: Use the output from the **Compose (Extract driveId)** action
   - **ID**: Use the output from the **Compose (Extract id)** action

Here's the flow structure:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extract driveId)**:
   - **Expression**:
     first(split(last(split(triggerOutputs()?['body/your_file_upload_field'], 'driveId=')), ''))
5. **Compose (Extract id)**:
   - **Expression**:
     first(split(last(split(triggerOutputs()?['body/your_file_upload_field'], 'id=')), ''))
6. **Get file content**:
   - **Drive ID**: Use the output from the **Compose (Extract driveId)** action
   - **ID**: Use the output from the **Compose (Extract id)** action

This approach should help you reliably extract the driveId and id parameters and use them to access the file content without encountering path errors.

Give this approach a try and let me know if it resolves the issue! If you need further assistance, feel free to ask.

You

Would this link need to be parsed by the OneDrive version of Get file content using path or the SharePoint version and what part exactly needs to be passed for it to work: https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B01D988B9-90D5-4F9E-8AAF-12E56C7FDEB5%7D&file=Case%201%20(4)_Akhtar%20Hasan.docx&action=default&mobileredirect=true

Copilot

To work with the provided link, you will need to use the SharePoint version of the **"Get file content using path"** action. Here's a step-by-step guide on how to parse the link and retrieve the file content:

1. **Extract the relevant part of the link**: The file path you need starts after the /personal/ part of the URL.
2. **Use the SharePoint connector**: Ensure you have the correct site address and list name.

Here's how you can extract the file path and use it in the **"Get file content using path"** action:

1. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
2. **Compose (Extracted Path)**: Add a **Compose** action to extract the relevant part of the file URL:
   - **Expression**:
     first(split(triggerOutputs()?['body/your_file_upload_field'], '?'))
     This expression will split the URL at the ? character and take the first part, giving you the path up to the query parameters.
3. **Get file content using path**:
   - **Action**: **Get file content using path** (SharePoint)
   - **Site Address**: Use your SharePoint site address
   - **File Path**: Use the output from the **Compose (Extracted Path)** action

Here's what the flow structure might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extracted Path)**:
   - **Expression**:
     first(split(triggerOutputs()?['body/your_file_upload_field'], '?'))
5. **Get file content using path** (SharePoint):
   - **Site Address**: Use your SharePoint site address
   - **File Path**: Use the output from the **Compose (Extracted Path)** action

This setup should correctly extract the file path from the URL and use it to retrieve the file content using the SharePoint connector.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

Would a string such as the following work in the Get file content using path (SharePoint) action?: personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B01D988B9-90D5-4F9E-8AAF-12E56C7FDEB5%7D&file=Case%201%20(4)_Akhtar%20Hasan.docx

Copilot

The string you provided includes encoded characters and query parameters that might not work directly in the **"Get file content using path"** (SharePoint) action. The SharePoint action typically requires a cleaner, direct path to the file within the document library.

The proper approach is to use the sourcedoc and file parameters to accurately locate the file path. However, the SharePoint action requires an exact path to the document, which isn't straightforward with the query parameters included in the URL.

Let's focus on obtaining the direct path to the document. Here's an updated approach:

1. **Extract the file parameter**: Retrieve the file name from the URL.
2. **Reconstruct the file path**: Combine it with the base document library path.

Here's how you can achieve this:

1. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
2. **Compose (Extract File Name)**: Add a **Compose** action to extract the file parameter from the URL:
   - **Expression**:
     first(split(last(split(triggerOutputs()?['body/your_file_upload_field'], 'file=')), '&'))
3. **Reconstruct the file path**: Combine the base path with the extracted file name.
4. **Get file content using path**:
   - **Action**: **Get file content using path** (SharePoint)
   - **Site Address**: Use your SharePoint site address
   - **File Path**: Construct the file path using your site's document library structure and the extracted file name.

Here's what the updated flow might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extract File Name)**:
   - **Expression**:
     first(split(last(split(triggerOutputs()?['body/your_file_upload_field'], 'file=')), '&'))
5. **Get file content using path** (SharePoint):
   - **Site Address**: Use your SharePoint site address
   - **File Path**: /sites/your-site-name/Shared Documents/ + output from **Compose (Extract File Name)**

Replace */sites/your-site-name/Shared Documents/* with the actual path to your SharePoint document library.

Give this refined approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

But the document isn't shared, it's a personal document uploaded from OneDrive, however the link given in teh file upload is always something like this:
https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B31FADAE3-8EA6-4297-8333-66B910530461%7D&file=9c%20Business%20Manager%20-%20CEO%20-%20Job%20profile_Akhtar%20Hasan.doc&action=default&mobileredirect=true

Copilot

Thank you for the clarification. Since the document is stored in a personal OneDrive location and the link is generated from a form submission, we need to retrieve the file contents from the user's personal OneDrive. Unfortunately, direct OneDrive file links in the format provided don't directly map to SharePoint paths.

Here's an approach to access the OneDrive file content by handling the provided URL:

1. **Extract the file path**: From the URL, get the relevant part of the path to access the file.
2. **Get file content from OneDrive**: Use the OneDrive connector to retrieve the file content.

Here's how to achieve this:

1. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
2. **Compose (Extract Drive Path)**: Add a **Compose** action to extract the relevant part of the URL:
   - **Expression**:
     replace(substring(outputs('Initialize_variable'), indexOf(outputs('Initialize_variable'), '/personal/'), sub(indexOf(outputs('Initialize_variable'), '&'),indexOf(outputs('Initialize_variable'), '/personal/'))), '%20', ' ')
3. **Get file content using path (OneDrive)**:
   - **Action**: **Get file content using path**
   - **File path**: Use the output from the **Compose (Extract Drive Path)** action.

Here's what the flow steps might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms).
2. **Get response details**: Use the response ID.
3. **Initialize variable**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
4. **Compose (Extract Drive Path)**:
   - **Expression**:
     replace(substring(outputs('Initialize_variable'), indexOf(outputs('Initialize_variable'), '/personal/'), sub(indexOf(outputs('Initialize_variable'), '&'), indexOf(outputs('Initialize_variable'), '/personal/'))), '%20', ' ')
5. **Get file content using path (OneDrive)**:
   - **File path**: Use the output from the **Compose (Extract Drive Path)** action.

By extracting the relevant part of the OneDrive URL and replacing the encoded characters, you should be able to use the OneDrive connector to retrieve the file content.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

Would an ID such as *01ZBAD6XHD3L5DDJUOS5BIGM3GXEIFGBDB* work in the Get file content action?

Copilot

Yes, an ID such as *01ZBAD6XHD3L5DDJUOS5BIGM3GXEIFGBDB* can indeed be used in the **"Get file content"** action with OneDrive or SharePoint. To properly use this ID in the action, you will need to ensure that you are using the correct connector and providing the appropriate context, such as the Drive ID or Site Address.

Here's how you can use this ID in the **"Get file content"** action:

1. **Initialize a variable to store the file ID**:
   - **Name**: fileID
   - **Type**: String
   - **Value**: *01ZBAD6XHD3L5DDJUOS5BIGM3GXEIFGBDB*
2. **Get file content**:
   - **Action**: **Get file content**

    - **Drive ID/Site Address**: Use the correct Drive ID (for OneDrive) or Site Address (for SharePoint).
    - **ID**: Use the fileID variable.

Here's the step-by-step flow structure:

1. **Trigger**: Use the appropriate trigger for your flow, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Use the response ID to retrieve the form details.
3. **Initialize variable**:
    - **Name**: fileID
    - **Type**: String
    - **Value**: *01ZBAD6XHD3L5DDJUOS5BIGM3GXEIFGBDB*
4. **Get file content**:
    - **Drive ID/Site Address**: Use the correct Drive ID or Site Address.
    - **ID**: Use the *fileID* variable.

For OneDrive:

- **Drive ID**: You can find the Drive ID in the file URL or by using OneDrive actions to list drives.
- **ID**: Use the *fileID* variable directly.

For SharePoint:

- **Site Address**: Use the URL of the SharePoint site.
- **File Path/ID**: Use the *fileID* variable directly.

This setup should allow you to retrieve the file content using the provided file ID. If you encounter any issues or need further assistance, feel free to ask!

You

How can I write this code as a compose formula? *newString[newString.find('id')+5:newString.index('type')-3]*

Copilot

To write this code as a Compose formula in Power Automate, you need to use string functions available in Power Automate to achieve the same result. Here's how you can convert the given Python-style code to a Compose formula:

The Python-style code:
newString[newString.find('id')+5:newString.index('type')-3]
Equivalent Compose formula in Power Automate:
substring(variables('newString'), add(indexOf(variables('newString'), 'id'), 5), sub(indexOf(variables('newString'), 'type'), add(indexOf(variables('newString'), 'id'), 8)))

Here's a step-by-step breakdown of what each part does:

1. **Index of 'id'**: indexOf(variables('newString'), 'id') finds the position of 'id' in the string.
2. **Starting position**: add(indexOf(variables('newString'), 'id'), 5) moves the start position to 5 characters after the 'id' substring.
3. **Index of 'type'**: indexOf(variables('newString'), 'type') finds the position of 'type' in the string.
4. **Ending position**: sub(indexOf(variables('newString'), 'type'), add(indexOf(variables('newString'), 'id'), 8)) calculates the length of the substring from the starting position to 3 characters before 'type'.

The final Compose formula combines these steps to extract the desired substring from the variable newString. Here's how you can use it in Power Automate:

1. **Initialize Variable**: (To store the newString)
    - **Name**: newString
    - **Type**: String
    - **Value**: (Your string value)
2. **Compose**: (To extract the substring)
    - **Expression**:
    - substring(variables('newString'), add(indexOf(variables('newString'), 'id'), 5), sub(indexOf(variables('newString'), 'type'), add(indexOf(variables('newString'), 'id'), 8)))

This formula will extract the substring starting 5 characters after 'id' and ending 3 characters before 'type' in the given string.

Give this approach a try and let me know if it works for you! If you need any more help, feel free to ask.

You

What would be the output of *substring(variables('newString'), add(indexOf(variables('newString'), 'id'), 5), sub(indexOf(variables('newString'), 'type'), add(indexOf(variables('newString'), 'id'), 8)))*
if it can't be found and how can I update *if(equals(indexOf(variables('fileID'), '%7B'), -1), 'Invalid ID', variables('fileID'))* to detect it if the above formula is stored as '*fileID*'?

Copilot

If the substring function cannot find the specified characters ('id' or 'type'), it will typically result in a runtime error or an invalid output because the positions calculated using indexOf will be -1.

To handle such cases, you should check for the existence of the required substrings before attempting to extract the substring. Here's how you can update your formula to handle this:

1. **Check if 'id' and 'type' exist in the string**:
2. **If they exist, extract the substring**:
3. **If not, return 'Invalid ID' or another appropriate message**:

Here's an updated version of your formula using nested if statements to detect the presence of 'id' and 'type' before extracting the substring:

1. Initialize variable (newString) to store the string.
2. Compose (Extract fileID):
    - **Expression**:

```
if(
  and(
    greaterOrEquals(indexOf(variables('newString'), 'id'), 0),
    greaterOrEquals(indexOf(variables('newString'), 'type'), 0)
  ),
  substring(
    variables('newString'),
    add(indexOf(variables('newString'), 'id'), 5),
    sub(indexOf(variables('newString'), 'type'), add(indexOf(variables('newString'), 'id'), 8))
  ),
  'Invalid ID'
)
```

Here's what the flow steps might look like:

1. **Initialize Variable**: (To store the newString)
    - **Name**: newString
    - **Type**: String
    - **Value**: (Your string value)
2. **Compose**: (To extract the fileID if valid)
    - **Expression**:

```
if(
  and(
    greaterOrEquals(indexOf(variables('newString'), 'id'), 0),
    greaterOrEquals(indexOf(variables('newString'), 'type'), 0)
  ),
  substring(
    variables('newString'),
    add(indexOf(variables('newString'), 'id'), 5),
    sub(indexOf(variables('newString'), 'type'), add(indexOf(variables('newString'), 'id'), 8))
  ),
  'Invalid ID'
)
```

This formula will check for the presence of 'id' and 'type' in *newString* before attempting to extract the substring. If either is not found, it will return 'Invalid ID'.

Give this approach a try and let me know if it works for you! If you need further assistance, feel free to ask.

You

Now these are the parameters for 'Get File content':

```
{
  "dataset": "https://bradfordgovuk.sharepoint.com",
  "id": "01ZBAD6XGYN5AHS4LV5ZA2V2A2UVWJSA5Q",
  "inferContentType": true
}
```

However I get the error:
Action 'Get_file_content' failed: File not found clientRequestId: adca3a49-0b62-4db2-8f9c-1ce3f5358a5d serviceRequestId: f1f484a1-30e3-b000-ce9e-a0393b7cb1c1

The full file upload that id is taken from is:
[{"name":"9c Business Manager - CEO - Job profile_Akhtar Hasan.doc","link":"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B31FADAE3-8EA6-4297-8333-66B910530461%7D&file=9c%20Business%20Manager%20-%20CEO%20-%20Job%20profile_Akhtar%20Hasan.doc&action=default&mobileredirect=true","id":"01ZBAD6XHD3L5DDJUOS5BIGM3GXEIFGBDB","type":null,"size":150528,"referenceId":"01ZBAD6XFVCUXPHDA7UVCYFLJ2YOJBTKNH","driveId":"b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0","status":1,"uploadSessionUrl":null}]

There is no Drive ID parameter in Get file content (both the OneDrive version and the sharePont version):

- The SharePoint version has Site Address & File Identifier
- The OneDrive version has File (The unique identifier of the file)

Copilot

Got it. Given the parameters available in the **"Get file content"** action, let's focus on using the **"Get file content"** action with the unique file identifier for either OneDrive or SharePoint.

For SharePoint:
1. **Site Address**: Use the SharePoint site address (e.g., https://bradfordgovuk.sharepoint.com).
2. **File Identifier**: Use the unique file identifier extracted from the form response.

For OneDrive:
1. **File Identifier**: Use the unique file identifier extracted from the form response.

Here's how you can update the flow to use these parameters:
1. **Trigger your flow**: Use the appropriate trigger, such as **"When a new response is submitted"** from Microsoft Forms.
2. **Get response details**: Add the **"Get response details"** action to retrieve the form submission details. Select the form and use the response ID from the trigger.
3. **Initialize a variable to store the file URL**:
   - **Name**: FileURL
   - **Type**: String
   - **Value**: Use the dynamic content from the file upload question.
4. **Compose (Extract File ID)**: Add a **Compose** action to extract the id parameter from the URL:
   - **Expression**:
     first(split(last(split(triggerOutputs()?['body/your_file_upload_field'], 'id=')), ''))
5. **Get file content using path (OneDrive)**:
   - **Action**: **Get file content**
   - **File Identifier**: Use the output from the **Compose (Extract File ID)** action
6. **Add file as an attachment** to the SharePoint list item if required.

Here's what the flow structure might look like:

1. **Trigger**: When a new response is submitted (Microsoft Forms)
2. **Get response details**: Use the response ID
3. **Initialize variable**:
    - **Name**: FileURL
    - **Type**: String
    - **Value**: Use the dynamic content from the file upload question
4. **Compose (Extract File ID)**:
    - **Expression**:
      first(split(last(split(triggerOutputs()?['body/your_file_upload_field'], 'id=')), ''))
5. **Get file content using path (OneDrive)**:
    - **File Identifier**: Use the output from the **Compose (Extract File ID)** action

This should allow you to retrieve the file content using the unique file identifier. If there are any specific parameters or additional steps you need to include, let me know! I'm here to help.

You

(Now Friday, Mar 7)
How can I setup an approvals system so that when the form is submitted by the requestor it sends an email to their manager (based on organisation structure) and lets them approve the request using Microsoft Approvals before it's then handled in the app?

Copilot

To set up an approval system that sends an email to the requestor's manager and integrates with Microsoft Approvals, you can use **Power Automate**. Here's a step-by-step guide to achieve this:

---

1. Trigger the Flow

- Use the **"When a new response is submitted"** trigger from **Microsoft Forms**.
- Select the form that the requestor will fill out.

---

2. Get Response Details

- Add the **"Get response details"** action to retrieve the form submission details.
- Use the **Response ID** from the trigger to fetch the details.

---

3. Identify the Manager

- Use the **"Get manager (V2)"** action from the **Office 365 Users** connector.
- Pass the requestor's email (from the form response) to this action to retrieve their manager's email address.

---

4. Create an Approval

- Add the **"Start and wait for an approval"** action from the **Approvals** connector.
- Configure the approval:
    - **Approval Type**: Choose "Approve/Reject – First to respond" or another type based on your needs.
    - **Title**: Provide a meaningful title for the approval request.
    - **Assigned To**: Use the manager's email retrieved in the previous step.
    - **Details**: Include relevant details from the form submission.
    - **Attachments**: If applicable, attach any files uploaded in the form.

---

5. Handle Approval Outcome

- Add a **Condition** to check the outcome of the approval:
    - **If Approved**:
        - Proceed with the next steps in your app or workflow (e.g., create a SharePoint list item, send a confirmation email, etc.).
    - **If Rejected**:
        - Notify the requestor via email about the rejection and include any comments from the manager.

---

6. Send Notifications

- Use the **"Send an email (V2)"** action from the **Outlook** connector to notify the requestor about the approval or rejection.

Example Flow Structure:

1. **Trigger**: When a new response is submitted (Microsoft Forms).
2. **Get Response Details**: Retrieve form submission details.
3. **Get Manager**: Use the requestor's email to find their manager.
4. **Start Approval**: Send an approval request to the manager.
5. **Condition**: Check the approval outcome.
    - **If Approved**: Proceed with the app or workflow.
    - **If Rejected**: Notify the requestor.

This setup ensures that the manager is notified and can approve or reject the request before it moves forward in your app. Let me know if you'd like help with any specific part of this process!

You

The Get manager connector doesn't return the email, it returns the following attributes:
About Me
Account enabled?
true if the account is enabled; otherwise, false. This property is required when a user is created.
Birthday
Business Phones Item
City
The city in which the user is located.
Company name
The name of the company in which the user works.
Country
The country/region in which the user is located; for example, "US" or "UK".
Department
The name of the department in which the user works.
Display Name
Given Name
Hire Date
Id
Interests Item
Job Title
Mail
Nickname
The mail alias for the user. This property must be specified when a user is created.
Mobile Phone
My Site
Office Location
Past Projects Item
Postal code
The postal code for the user's postal address. The postal code is specific to the user's country/region. In the United States of America, this attribute contains the ZIP code.
Preferred Language
Preferred Name
Responsibilities Item
Schools Item
Skills Item
State
Street Address
Surname
User Principal Name
User Type

Copilot

Thank you for pointing that out! The **Get manager** (V2) connector does indeed provide detailed attributes, and the one you'll likely need for sending the approval email is the **Mail** attribute or the **User Principal Name** (UPN), as these can be used as email addresses in most Office 365 environments.

Here's how you can update the flow to use the manager's email:

Updated Approval Flow Setup:

1. **Trigger**: Use the **"When a new response is submitted"** trigger from **Microsoft Forms**.
2. **Get Response Details**: Add the **"Get response details"** action to retrieve the submitted form details.
3. **Get Manager (V2)**: Add the **"Get manager (V2)"** action to fetch the manager's details:
    - Use the form responder's **email** or **User Principal Name (UPN)** as input to this action.
4. **Extract Manager's Email**:
    - The manager's email is available in the **Mail** field returned by the **Get manager (V2)** action. Use this directly for sending the approval request.
5. **Start Approval**:
    - Add the **"Start and wait for an approval"** action.
    - Configure:
        - **Approval Type**: "Approve/Reject – First to respond"
        - **Title**: Provide a meaningful title for the approval request.
        - **Assigned To**: Use the **Mail** or **User Principal Name (UPN)** from the **Get manager (V2)** action.
        - **Details**: Include any relevant information from the form response.
6. **Condition**:
    - Add a condition to check the result of the approval (Approved or Rejected):
        - **If Approved**: Proceed with the next steps (e.g., update records, notify the requestor, etc.).
        - **If Rejected**: Notify the requestor with the rejection reason.
7. **Notifications**:
    - Use **"Send an email (V2)"** to notify the requestor of the approval status.

Key Notes:

- The **Mail** attribute provided by **Get manager (V2)** corresponds to the manager's email and should be used in the **Assigned To** field of the approval.
- If the **Mail** field is empty in your environment (some setups may omit this), you can use **User Principal Name (UPN)** as a fallback, as it also functions as an email address in most cases.

With this configuration, the approval request will be sent to the manager, and their decision will drive the next steps in your flow. Let me know if you run into any challenges or need further assistance!

You

What should I put in the details based on the fact that this email is sent to the requestor?
Hi @{outputs('Get_user_profile_(V2)')?['body/givenName']},
Your SAP Request has been received.
It will be processed by the SAP Champions and Finance in due time.

The Details of your request were as follows
Request Date: @{triggerBody()?['RequestDate']}
Requestor Email: @{triggerBody()?['RequestorName']}
Requestor Department: @{triggerBody()?['RequestorDepartment']}
Request Urgency: @{triggerBody()?['RequestUrgency']}
Basic Info: @{triggerBody()?['BasicRequestInfo']}
Additional Info: @{triggerBody()?['AdditionalInfo']}
Requestor Name: @{triggerBody()?['RequestorManagerName']}

This email has been auto-generated, please do not reply

Copilot

This email already seems well-structured and informative. However, here are a few suggestions to make it more clear and relevant for the requestor while keeping the details concise and professional:

Recommended Adjustments:

1. **Clarity on "Requestor Name"**:
   - The last line in the details (Requestor Name: @{triggerBody()?['RequestorManagerName']}) seems to be referencing the manager's name. Instead, you could clarify this field as "Manager Name" or "Requestor's Manager" for better context.
2. **Add Confirmation or Next Steps**:
   - To reassure the requestor, you could specify what happens next (e.g., "Your request has been forwarded to your manager for approval.").
3. **Polish the Language**:
   - Slightly rephrase the opening and closing messages for a more natural and friendly tone.

Updated Email Template:

Hi @{outputs('Get_user_profile_(V2)')?['body/givenName']},

Thank you for submitting your SAP Request. It has been successfully received and will be processed by the SAP Champions and Finance team in due course.

Here are the details of your request for reference:

*- **Request Date**: @{triggerBody()?['RequestDate']}*
*- **Requestor Email**: @{triggerBody()?['RequestorName']}*
*- **Department**: @{triggerBody()?['RequestorDepartment']}*
*- **Urgency Level**: @{triggerBody()?['RequestUrgency']}*
*- **Basic Request Info**: @{triggerBody()?['BasicRequestInfo']}*
*- **Additional Details**: @{triggerBody()?['AdditionalInfo']}*
*- **Manager**: @{triggerBody()?['RequestorManagerName']}*

Your request is now under review. If any further information is needed, someone from the team will reach out to you.

This email has been auto-generated. Please do not reply.

Best regards,
[Your Organization's Name or SAP Request Team]

Key Improvements:

- **"Manager" Label**: Renamed the ambiguous "Requestor Name" field to "Manager" for clarity.
- **Friendly Opening and Closing**: Enhanced the tone to feel more natural and approachable.
- **Next Steps**: Added a sentence to explain what happens after submission, ensuring the requestor is informed.

You can tailor the tone further based on your organizational style! Let me know if you'd like to refine this even more.

You

No I mean how can I use this email for the basis of the request details in the approval that is going to the manager of the requestor and adapt it for the new use case?

Copilot

To adapt this email for a request approval scenario sent to the manager, we can modify the details slightly and frame the message as an official request for approval. The goal is to provide enough context for the manager to understand the request, along with an actionable prompt for them to either approve or reject it.

Here’s how the adapted approval request could look:

Adapted Approval Details for Manager:

Hi @{outputs('Get_manager_(V2)')?['body/givenName']},

You have a new SAP Request from your team member requiring your approval.

The details of the request are as follows:

*- **Request Date**: @{triggerBody()?['RequestDate']}*
*- **Requestor Email**: @{triggerBody()?['RequestorName']}*
*- **Requestor Department**: @{triggerBody()?['RequestorDepartment']}*
*- **Request Urgency**: @{triggerBody()?['RequestUrgency']}*
*- **Basic Info**: @{triggerBody()?['BasicRequestInfo']}*
*- **Additional Info**: @{triggerBody()?['AdditionalInfo']}*
*- **Requestor Manager**: @{triggerBody()?['RequestorManagerName']}*

Please review the details and respond to this request using the Approve or Reject options provided.

This email has been auto-generated. If you have further questions, please reach out directly to the SAP team or the requestor.

Thank you,
[Your Organization's Name or SAP Request Team]

Key Changes:

1. **Address the Manager**:
   - Replaced the recipient's reference with the manager's name (e.g., from the Get Manager (V2) connector).
2. **Reframe the Message for Action**:
   - Changed the tone to focus on the manager's responsibility to approve or reject the request.
   - Added a clear call to action: "Please review the details and respond to this request using the Approve or Reject options provided."
3. **Provide Context**:
   - Highlighted that the request comes from one of their team members.
   - Included fields like "Requestor Email" and "Requestor Department" for easy reference.
4. **Clarify Follow-Up**:
   - Added a note about reaching out to the SAP team or requestor for further clarification.

This updated message is ready to be used as the **Details** field in the **"Start and wait for an approval"** action. It will provide the manager with all necessary context and prompt them to make an informed decision. Let me know if there's anything else you'd like to refine.

You

There are 2 more boxes in the approval connection, how should I fill these out?
Item link
Add a link to the item to approve
Item link description
Describe the link to the item

Copilot

The **"Item link"** and **"Item link description"** fields are optional in the approval connector, but filling them out properly can enhance the approval process by providing more context. Here's how to use them effectively:

1. Item link:

- This should be a direct link to the item or request that is being approved. For example:
  - If you're storing the request in **SharePoint**, this would be the link to the specific SharePoint list item.

- If the request involves an uploaded document stored in **OneDrive** or **SharePoint**, this could be the link to the document.
- If you have no specific stored item but are using the form submission as the "item," you can provide the link to the submitted form (if accessible) or wherever the request details are stored.

Example:
https://bradfordgovuk.sharepoint.com/sites/YOUR_SITE/Lists/YOUR_LIST/DispForm.aspx?ID={Dynamic ID from Create Item or Form Submission}
Use the dynamic content available in your flow to insert the appropriate link.

---

2. Item link description:

- This is a short description of the link to let the manager understand what the link is about.
- Keep it brief and relevant. For example:
  - "View request details"
  - "Open the document associated with this request"
  - "View the SharePoint item for this request"

Example:
View the SAP Request details submitted by the requestor.

How to Populate These Fields in Your Flow:

1. **Item Link**:
   - Use dynamic content from the flow where the item or document is stored.
   - For example:
     - If you're creating a SharePoint list item for the request, use the dynamic "Link to item" from the **Create Item** action.
     - If you're attaching a file, use the file's URL from **OneDrive** or **SharePoint**.
2. **Item Link Description**:
   - Write a concise and informative description for the manager, explaining what they will see when they click the link.

Example Configuration:

- **Item link**:
- *https://bradfordgovuk.sharepoint.com/sites/SAPRequests/Lists/Requests/DispForm.aspx?ID=@{outputs('Create_item')?['body/ID']}*
- **Item link description**:
- View SAP request details submitted by *@{triggerBody()?['RequestorName']}*.

By filling these fields correctly, you'll provide the manager with quick access to all necessary information and context, making the approval process seamless. Let me know if you'd like help refining the flow further!

You

How should I set up the condition for checking approval?

Copilot

To set up a condition to check the outcome of an approval process in Power Automate, you can use the **"Outcome"** field from the **"Start and wait for an approval"** action. This field will provide the result of the approval, such as "Approve" or "Reject." Here's how to configure it step by step:

Steps to Set Up the Approval Condition

1. **Add the Approval Action**:
   - Use the **"Start and wait for an approval"** action to initiate the approval process.
   - Ensure that the request is configured properly, with the manager as the approver.
2. **Condition to Check the Approval Outcome**:
   - After the approval action, add a **Condition** to evaluate the approval outcome.
   - In the condition:
     - **Field**: Select the **Outcome** dynamic content from the **Approval** action.
     - **Operator**: Choose is equal to.
     - **Value**: Enter Approve.
3. **Configure the Outputs of the Condition**:
   - **If Yes (Approved)**:
     - Add the actions you want to perform when the request is approved (e.g., notify the requestor, move the request to the next step in the process, create a record in a database, etc.).
   - **If No (Rejected)**:
     - Add the actions for handling rejected requests (e.g., notify the requestor about the rejection and include any comments provided by the approver).

Example Flow Structure:

1. **Trigger**:
   - When a new response is submitted (e.g., from Microsoft Forms).
2. **Get Manager**:
   - Use the **"Get Manager (V2)"** action to determine the approver.
3. **Start and Wait for Approval**:
   - Assign the approval request to the manager using their email or UPN.
   - Set the approval type (e.g., "Approve/Reject – First to respond").
   - Include the request details for context.
4. **Condition**:
   - **Field**: Outcome
   - **Operator**: is equal to
   - **Value**: Approve
   - **If Yes**:
     - Take actions such as:
       - Notify the requestor of approval.
       - Proceed with the next steps in your workflow.
   - **If No**:
     - Take actions such as:
       - Notify the requestor of rejection.
       - Provide details about the rejection (if available via approver comments).

Example in a Condition:

Field: outputs('Start_and_wait_for_an_approval')?['body/outcome']
Operator: is equal to
Value: Approve

Key Notes:

- If you want to include the comments provided by the approver (if any), you can use the **"Comments"** field from the approval action and include it in your notification to the requestor.
- Ensure your approval action has all necessary details (e.g., item links, descriptions) for a smooth decision-making process.

Let me know if you'd like help building this step or further refining the workflow!

You

(Now Tuesday, Mar 11)
How would I call a flow with multiple parameters in power apps, the flow is called "Send email based on request status" and I want to activate it when a button is pressed

Copilot

To call a flow with multiple parameters in Power Apps, you'll need to integrate the flow into your app and pass the required parameters through the button's OnSelect property. Here's how you can set it up:

Steps to Call a Flow with Multiple Parameters:

1. Add the Flow to Your Power Apps App:
- In Power Apps Studio, go to the "Action" menu in the top navigation bar.
- Select **"Power Automate"**.
- Find the flow called **"Send email based on request status"** and add it to your app. This links the flow to your app.

---

2. Understand the Flow Parameters:
- Make sure your flow is set up with input parameters (e.g., email, status, name, etc.).
- Note the exact names of the parameters in the flow, as you'll need these to pass values from Power Apps.

---

3. Set Up the Button in Power Apps:
- Add a button to your app (or use an existing button).
- Go to the **OnSelect** property of the button and call the flow using the formula:
- 'Send email based on request status'.Run(Parameter1, Parameter2, Parameter3, ...)
- Replace Parameter1, Parameter2, etc., with the actual values or variables you want to pass to the flow.

---

4. Example:

- Suppose your flow has three input parameters: email, status, and name.
- Your formula for the button's OnSelect property would look like this:

```
'Send email based on request status'.Run(
    TextInputEmail.Text,
    DropdownStatus.Selected.Value,
    TextInputName.Text
)
```

- Here's what each part does:
  - TextInputEmail.Text: The email address entered in a text input field.
  - DropdownStatus.Selected.Value: The selected value from a dropdown control.
  - TextInputName.Text: The name entered in another text input field.

---

5. Run and Test:
- Save and publish your app.
- Press the button to trigger the flow and ensure the parameters are passed correctly.

Things to Keep in Mind:

- **Ensure Flow Integration**: Verify that the flow is correctly linked to your app and is accepting the parameters in the correct order.
- **Error Handling**: You can use IfError in Power Apps to handle any errors that occur when calling the flow.

```
IfError(
    'Send email based on request status'.Run(Parameter1, Parameter2, Parameter3),
    Notify("An error occurred while triggering the flow", NotificationType.Error)
)
```

- **Data Types**: Ensure that the data types of the parameters in Power Apps match those expected by the flow.

Let me know if you want help with configuring a specific part of this setup!

You

I currently have this if statement in power apps, how can I make it so that when the page loads and the if statement is true, it will bring if up a confirm/cancel pop-up box?
If(!IsBlank(Param("approved")) && Gallery1_1.Selected.Title = requestID && !fromPage, "This request is approved!", "None")

Copilot

To display a confirm/cancel pop-up box in Power Apps when the page loads and the If statement is true, you can use a combination of a **variable**, the OnVisible property of the screen, and a **Dialog Box** component created using a container or card. Here's how you can set it up:

Step-by-Step Solution:

1. Create a Variable for the Pop-Up:

- You need a variable to control the visibility of the pop-up.
- When the page loads, check your If condition, and if it's true, set the variable to true to display the pop-up.

2. Set Up the OnVisible Property:

- Go to the screen where you want this behavior to occur.
- Set the OnVisible property of the screen to something like:

```
If(
    !IsBlank(Param("approved")) && Gallery1_1.Selected.Title = requestID && !fromPage,
    UpdateContext({ShowPopup: true}),
    UpdateContext({ShowPopup: false})
)
```

- This will check the condition when the screen becomes visible and set the ShowPopup variable to either true or false.

3. Create the Confirm/Cancel Pop-Up:

- Add a **Container** or **Card** to your screen to act as the pop-up box.
- Include a label or text box to show the message (e.g., "This request is approved!") and two buttons for **"Confirm"** and **"Cancel"**.
- Set the **Visible** property of the container or card to the ShowPopup variable:
- ShowPopup

4. Configure the Buttons:

- **Confirm Button**:
    - Set the OnSelect property to handle the confirmation action (e.g., navigate to another screen, update a value, or any desired logic).
    - Example:
    - UpdateContext({ShowPopup: false});
    - // Add your logic for Confirm action here
- **Cancel Button**:
    - Set the OnSelect property to simply hide the pop-up.
    - Example:
    - UpdateContext({ShowPopup: false})

5. Test the Functionality:

- When the page loads and the condition in the If statement evaluates to true, the ShowPopup variable will be set to true, making the pop-up visible.
- The user can then either confirm or cancel, and the appropriate actions will be triggered.

Example Code for Your Scenario:

OnVisible Property of the Screen:

```
If(
    !IsBlank(Param("approved")) && Gallery1_1.Selected.Title = requestID && !fromPage,
    UpdateContext({ShowPopup: true}),
    UpdateContext({ShowPopup: false})
)
```

Confirm Button OnSelect Property:

```
UpdateContext({ShowPopup: false});
// Add logic for confirmation
Notify("Request approved", NotificationType.Success);
```

Cancel Button OnSelect Property:

```
UpdateContext({ShowPopup: false});
Notify("Action canceled", NotificationType.Warning);
```

Container/Card Visible Property:

```
ShowPopup
```

This approach ensures that the pop-up only appears when the If condition is true, and it gives users the option to confirm or cancel their action. Let me know if you'd like further assistance with any of these steps!

Conversation by Fifa Utuber with Copilot on Tuesday, Feb 4 2025 (to Tuesday, Mar 11)